

AI-Assisted Treasury Bond Analytics Validation

Correcting and validating a `USTreasurySecurity` class for fixed-income analytics

This report compares two AI-generated implementations of a Python `USTreasurySecurity` class and documents the corrected final version. The class supports U.S. Treasury quote parsing, actual/actual accrued interest, dirty price, Newton-solved yield to maturity, PV01, modified duration, Macaulay duration, and convexity. The goal was not only to produce working code, but to test whether apparent agreement between two AI systems was enough to establish correctness.

Key conclusion: Agreement between two AI-generated solutions is useful, but it is not a substitute for checking the original specification. Both drafts passed standard benchmark tests yet shared the same quote-parsing error for the 100-24++ convention.

Project Scope

The assignment required one Python file with a `USTreasurySecurity` class. Inputs included a Treasury clean-price quote, remaining coupon periods, annual coupon rate, trade date, previous and next coupon dates, and a holiday calendar for next-business-day settlement. Outputs had to be returned as floats, with prices as percent of par and yields or coupon rates quoted in percent.

- Clean price and dirty price using Treasury 32nds quote conventions.
- Actual/actual accrued interest between coupon dates.
- Standard between-coupon dirty-price discounting with $(1 + y/2)^{(i + z/x)}$.
- Simple-interest discounting when only one coupon remains.
- YTM solved with Newton's method at tolerance $1e-14$.
- PV01, modified duration, Macaulay duration, and convexity with explicit annualized-yield units.

Comparison of AI Drafts

GPT-5.5 and Claude Opus 4.7 produced broadly similar implementations and matched standard bond-pricing benchmarks to high precision. Their differences were still semantically important because they affected edge cases, convergence behavior, validation strictness, and the independence of risk-measure calculations.

Area	Draft behavior	Risk	Final correction
Quote parsing	Both drafts treated 100-24++ as adding only 1/128 beyond 24/32.	Clean price, dirty price, YTM, and risk measures are all shifted for ++ quotes.	Parse ++ cumulatively as $1/64 + 1/128$, matching the course slide convention.
Empty fractional quote	GPT-5.5 accepted 100-; Claude raised an error.	A valid quote-like edge case could fail unnecessarily.	Treat the empty fractional part as zero.
Newton initialization	Neither draft started from $y_0 = 0$.	The solution could diverge from the guaranteed lecture setup.	Start Newton's method at $y_0 = 0$.
PV01 units	One draft used <code>abs()</code> ; the other relied on derivative sign.	Sign and unit conventions were easy to misread.	Use $-dP/dy * 1e-4$ and document annualized decimal-yield units.

Area	Draft behavior	Risk	Final correction
Validation and caching	The drafts differed in settlement-window validation and repeated YTM solves.	Edge inputs and repeated calls could behave differently.	Keep stricter validation and cache the solved YTM.

Testing Strategy

Testing was layered instead of relying on one set of happy-path benchmarks. The two drafts were first checked against the five Treasury bonds in the benchmark workbook and a lecture example. After that, the code was read line by line to identify behavioral differences, and targeted probes were created for edge cases such as empty fractional quotes and cumulative plus-suffix parsing.

- The benchmark cases confirmed the standard pricing path but did not expose all specification errors.
- The slide-6 analytical example confirmed the chain-rule conversion from per-period rate to annualized yield.
- The 100-24++ probe exposed the shared bug that normal benchmarks missed.
- The corrected implementation passes the benchmark cases, parses 100-24++ as 100.7734375, accepts 100-, and converges from $y_0 = 0$ across the tested securities.

Corrected Implementation

The final version uses the Claude draft as a structural base because its private-method organization, stricter validation, and YTM caching were cleaner. It borrows GPT-5.5's explicit handling of empty fractional quotes, then applies the specification-level corrections that neither draft fully captured.

- Quote parsing now handles + and ++ as cumulative Treasury fractional adjustments.
- Date handling uses next-business-day settlement and validates the settlement position within the current coupon period.
- Pricing, first derivative, and second derivative are written with explicit annualized-yield and per-period-yield conversions.
- PV01 is defined as the positive price change for a one-basis-point decrease in annualized yield.

Takeaway

The main lesson is that AI-generated code can be productive when it is treated as a draft, not as authority. Two independent-looking models can still share the same mistake when the prompt is ambiguous. The corrected project therefore combines model comparison, benchmark testing, edge-case probing, and direct specification review into one validation workflow.